

TJSC 2026 - Analista de Sistemas

Arquitetura de Sistemas e Integração

Aula 8 - Mensageria, eventos comerciais, Message Broker e RabbitMQ

Material autoral de estudo - padrão FGV - versão completa, sem flashcards

Foco: transformar vivência prática em APIs, Java/Quarkus, Angular e integrações corporativas em acerto de prova.

Recorte da aula

Esta aula cobre a Aula 8 da trilha: mensageria, eventos comerciais, Message Broker e RabbitMQ. No edital, o foco direto são os itens 1.13 - Mensageria e eventos comerciais e 1.14 - Message Broker e RabbitMQ. O tema conversa com microsserviços, integração de sistemas, PDPJ, API Gateway, Service Discovery e ambientes distribuídos, mas sem transformar a aula em tutorial de framework.

1. Identificação da aula e aderência ao edital

A Aula 8 trata da comunicação assíncrona entre sistemas, especialmente por mensagens e eventos. Em prova da FGV, esse assunto raramente aparece como mera definição decorada. A banca tende a criar cenários em que o candidato precisa decidir se faz sentido usar chamada REST síncrona, fila, evento comercial, broker, webhook, retry, dead letter queue ou integração direta.

Objetivo de prova: diferenciar mensageria de API REST, evento de comando, broker de gateway, fila de exchange, produtor de consumidor, publish/subscribe de work queue, confirmação de consumo de confirmação de publicação, e saber quando RabbitMQ ajuda ou aumenta complexidade.

Item do edital/trilha	Cobertura nesta aula	Foco FGV
1.13 Mensageria e eventos comerciais	mensagem, evento, comando, fila, tópico, producer, consumer, coreografia, consistência eventual, idempotência, retries e DLQ	distinguir evento comercial de chamada síncrona e de notificação informal
1.14 Message Broker e RabbitMQ	broker, exchange, queue, binding, routing key, ack/nack, publisher confirm, TTL, DLX, tipos de exchange e fluxo AMQP	separar papel do broker dos papéis de API Gateway, Service Discovery, banco e ESB
Conexões instrumentais	microsserviços, integração corporativa, PDPJ, arquitetura distribuída, Angular/Java/Quarkus/Spring	ler cenários reais sem confundir tecnologia com decisão arquitetural

Estratégia FGV

A FGV costuma cobrar alternativas quase certas: "RabbitMQ garante exatamente uma entrega", "fila resolve transação distribuída", "evento é comando", "DLQ corrige mensagem inválida", "broker substitui API Gateway" ou "mensageria é sempre melhor que REST". O acerto vem de reconhecer limite, trade-off e responsabilidade de cada componente.

2. Vocabulário essencial

Termo	Definição objetiva	Pegadinha FGV
Mensagem	Unidade de dados enviada de um produtor para um broker ou consumidor. Pode representar comando, evento, notificação ou tarefa.	Achar que toda mensagem é evento comercial.
Evento comercial	Fato relevante que já ocorreu no domínio, como ProcessoDistribuido, DocumentoJuntado ou PagamentoCustasConfirmado.	Confundir evento com comando; evento descreve fato passado.

Comando	Intenção de executar ação, como EmitirNotificacao ou GerarGuiaCustas.	Tratar comando como fato já ocorrido.
Producer / Publisher	Aplicação que publica mensagem ou evento no broker.	Achar que produtor escolhe diretamente todos os consumidores.
Consumer / Subscriber	Aplicação que recebe e processa mensagens de uma fila ou inscrição.	Achar que consumidor sempre é frontend ou usuário final.
Message Broker	Intermediador que recebe, roteia, armazena temporariamente e entrega mensagens conforme regras.	Confundir com API Gateway, banco de dados ou Service Discovery.
Queue	Fila onde mensagens ficam até serem entregues/consumidas.	Achar que fila é o mesmo que exchange.
Exchange	Componente RabbitMQ que recebe publicações e as roteia para filas conforme tipo, bindings e routing key.	Publicar diretamente na fila em todos os cenários; no modelo AMQP, publica-se em exchanges.
Binding	Ligação entre exchange e fila, eventualmente com chave/padrão de roteamento.	Confundir binding com consumer.
Routing key	Chave textual usada pelo exchange para decidir para quais filas a mensagem deve ir.	Confundir com chave primária de banco.
Ack / Nack	Confirmação positiva ou negativa do consumidor sobre o processamento da mensagem.	Achar que ack do consumidor confirma que o produtor publicou com sucesso.
Publisher confirm	Confirmação do broker ao produtor de que a publicação foi aceita pelo broker.	Confundir com ack de consumo.
DLQ / DLX	Dead Letter Queue/Exchange: destino para mensagens rejeitadas, expiradas ou que excederam tentativas.	Achar que DLQ conserta automaticamente erro de negócio.
TTL	Time To Live: tempo máximo de vida de mensagem ou fila.	Achar que TTL é garantia de entrega.
Prefetch	Limite de mensagens não confirmadas entregues a um consumidor.	Confundir com paginação de API REST.

3. Mensageria: o que resolve e o que não resolve

Mensageria é um padrão de integração no qual sistemas se comunicam por mensagens intermediadas por um componente especializado, o broker. O produtor não precisa chamar diretamente o consumidor no mesmo instante. Isso permite desacoplamento temporal: o produtor publica e segue seu fluxo; o consumidor processa quando puder, de acordo com disponibilidade, escala, prioridade e políticas de retry.

Esse desacoplamento é valioso em ambientes distribuídos, mas não é gratuito. Ele adiciona infraestrutura, observabilidade, contratos de mensagem, idempotência, tratamento de duplicidade, monitoramento de fila, políticas de retry e responsabilidade sobre mensagens que falham.

Quando mensageria costuma ajudar	Por quê
Processamento demorado	O usuário não precisa esperar a execução inteira; a aplicação pode responder "aceito para processamento".
Picos de demanda	A fila suaviza carga e permite consumidores processarem no ritmo suportado.
Integração entre serviços independentes	Produtor e consumidor ficam menos acoplados em tempo, disponibilidade e endereço físico.
Eventos relevantes para múltiplos interessados	Um fato comercial pode ser consumido por auditoria, notificação, BI, integração externa etc.
Retries controlados	Falhas transitórias podem ser reprocessadas sem depender de usuário ou cron manual.

Quando mensageria pode ser escolha ruim	Risco
Consulta simples que exige resposta imediata	REST/HTTP síncrono tende a ser mais simples e transparente.

Fluxo que precisa de consistência transacional forte imediata entre múltiplos recursos	Fila não "mágica" resolve transação distribuída.
Equipe sem observabilidade ou operação do broker	Mensagens podem acumular, duplicar, expirar ou ir para DLQ sem tratamento.
Eventos sem contrato e sem versionamento	Consumidores quebram quando payload muda.
Uso para esconder baixa qualidade de domínio	Mensageria não substitui modelagem correta, idempotência e regras claras.

Exemplo com seu contexto

Em uma tela Angular, o usuário solicita geração de relatório pesado. Se a API Java/Quarkus tentar gerar tudo dentro da mesma request HTTP, a chamada pode expirar. Uma solução mais robusta é a API gravar a solicitação, publicar uma mensagem GeracaoRelatorioSolicitada e retornar 202 Accepted. Um consumidor processa, gera o arquivo e publica RelatorioGerado. A tela consulta status ou recebe notificação. Isso é arquitetura assíncrona; não é simplesmente "colocar RabbitMQ em tudo".

4. Comunicação síncrona x assíncrona

Critério	Síncrona - REST/HTTP típico	Assíncrona - Mensageria/eventos
Tempo de resposta	Cliente espera resposta na mesma chamada.	Produtor não precisa aguardar o processamento final.
Acoplamento temporal	Maior: consumidor precisa estar disponível no momento da chamada.	Menor: mensagem pode aguardar consumidor.
Simplicidade operacional	Mais simples para consulta direta e CRUD comum.	Exige broker, filas, monitoramento, retries e DLQ.
Falhas	Timeout/erro visível para o chamador.	Falha pode ser reprocessada ou acumulada em fila/DLQ.
Rastreabilidade	Mais direta por request/response.	Exige correlation ID, logs distribuídos e tracing.
Consistência	Tende a favorecer resposta imediata do estado atual.	Frequentemente trabalha com consistência eventual.
Uso típico	Consulta de processo por número, busca de usuário, validação imediata.	Notificação, auditoria, atualização de projeção, geração pesada, integração com sistema instável.

Regra de prova

REST síncrono não é "errado" e mensageria não é "sempre melhor". O arquiteto escolhe pelo requisito: latência, acoplamento, consistência, carga, rastreabilidade, criticidade e tolerância a processamento posterior.

5. Eventos negociais: fato ocorrido, não pedido de ação

Evento negocial é um fato relevante do domínio que já aconteceu e interessa a outros componentes. O nome geralmente aparece no particípio ou passado: ProcessoDistribuido, AudienciaDesignada, DocumentoAssinado, CustasPagas. A ideia é que o produtor informe uma mudança de estado, e os consumidores decidam o que fazer com esse fato.

Tipo	Expressa	Exemplo	Erro comum
Evento	Fato ocorrido no domínio.	ProcessoDistribuido, DocumentoJuntado.	Usar evento para mandar outro serviço "fazer agora".
Comando	Pedido/intenção de ação.	EnviarEmail, GerarMandado.	Nomear comando como se fosse evento.
Consulta	Pedido de leitura de estado.	GET /processos/{id}.	Colocar consulta simples em fila sem necessidade.
Notificação	Comunicação para usuário/sistema sobre algo.	Avisar parte sobre audiência.	Confundir notificação com fonte da verdade do evento.

A distinção importa porque eventos favorecem coreografia: cada serviço reage a fatos relevantes sem depender de um orquestrador central. Em compensação, a coreografia exige disciplina: contrato de evento, versionamento, observabilidade, idempotência e cuidado com efeitos colaterais.

Estilo	Como funciona	Vantagem	Risco
Coreografia	Serviços publicam e reagem a eventos; o fluxo global emerge das reações locais.	Baixo acoplamento e boa autonomia.	Fluxo difícil de rastrear sem observabilidade.
Orquestração	Um coordenador central comanda etapas e chama serviços.	Visão explícita do processo.	Coordenador pode virar gargalo/monólito de processo.
Request/response	Cliente chama serviço e espera resposta.	Simples e previsível para consultas e validações imediatas.	Acoplamento temporal e risco de cascata de falhas.

6. Message Broker: papel arquitetural

Message Broker é o intermediador de mensagens. Ele recebe mensagens de produtores, aplica regras de roteamento, armazena em filas quando necessário e entrega a consumidores. Em arquitetura, ele reduz o acoplamento direto entre sistemas, mas não elimina contrato, semântica de negócio e governança.

Componente	Responsabilidade	Não confundir com
Message Broker	Recebe, roteia, enfileira e entrega mensagens/eventos.	API Gateway, banco de dados, Service Discovery.
API Gateway	Entrada HTTP, roteamento de APIs e políticas transversais.	Broker de eventos.
Service Discovery	Registro/localização dinâmica de serviços.	Fila ou exchange.
Banco de dados	Persistência transacional do estado de negócio.	Fila durável de integração.
ESB	Barramento de integração mais amplo, frequentemente com transformação/orquestração central.	Broker leve usado apenas para mensagens.

Ponto FGV

Se a alternativa disser que o broker autentica usuários finais, descobre instâncias, renderiza telas Angular ou substitui o banco transacional, desconfie. Broker é sobre mensagens, roteamento assíncrono e entrega a consumidores.

7. RabbitMQ: visão geral para prova

RabbitMQ é um broker de mensageria maduro e amplamente utilizado, com suporte a protocolos abertos e múltiplas bibliotecas clientes. Para o edital, o foco não é decorar comandos de instalação, mas entender o modelo AMQP: produtores publicam mensagens em exchanges; exchanges roteiam para filas; consumidores recebem mensagens das filas.

Produtor / Publisher

|

| publica mensagem com exchange + routing key

v

Exchange -- binding/routing key --> Queue --> Consumer

Peça RabbitMQ	Função	Exemplo mental
Producer	Publica mensagem.	API de processos publica DocumentoJuntado.
Exchange	Recebe publicação e decide para quais filas encaminhar.	Exchange eventos.processos.
Queue	Armazena mensagens até consumo.	fila.notificacao, fila.auditoria.

Binding	Regra que liga exchange à fila.	fila.auditoria recebe eventos processo.#.
Routing key	Chave usada no roteamento.	processo.documento.juntado.
Consumer	Processa mensagens da fila.	Serviço de notificação, auditoria, indexação.
Connection	Conexão TCP entre aplicação e broker.	Aplicação Java abre conexão com RabbitMQ.
Channel	Canal lógico dentro de uma conexão.	Operações AMQP ocorrem em canais.
Virtual host	Isolamento lógico dentro do broker.	Ambientes ou domínios separados por vhost.

8. Exchanges, filas, bindings e routing keys

No RabbitMQ, **a exchange é a peça que recebe a publicação**. A fila é o local onde a mensagem fica aguardando consumo. A binding é a associação entre exchange e fila. **A routing key ajuda o exchange a decidir o destino**. A FGV pode trocar esses papéis.

Tipo de exchange	Como roteia	Uso típico	Pegadinha
Direct	Encaminha para filas cuja binding key casa exatamente com a routing key.	Comandos ou eventos com destino por chave exata.	Achar que direct envia para todos.
Fanout	Copia a mensagem para todas as filas ligadas à exchange, ignorando routing key.	Broadcast para múltiplos consumidores independentes.	Achar que fanout filtra por chave.
Topic	Roteia por padrões na routing key, usando * e #.	Eventos por domínio: processo.*.juntado, processo.#.	Confundir * com #.
Headers	Roteia com base em cabeçalhos da mensagem.	Cenários com critérios por metadados, não por string de routing key.	Achar que é o tipo mais comum para tudo.
Default exchange	Exchange sem nome que roteia para fila com routing key igual ao nome da fila.	Casos simples de publicação direta por nome da fila.	Esquecer que ainda existe uma exchange no modelo.

Memorização útil

Direct = chave exata. Fanout = broadcast. Topic = padrão com pontos, * e #. Headers = metadados. Default = fila pelo próprio nome.

9. Entrega, confirmações e confiabilidade

Mensageria confiável não nasce apenas de “usar RabbitMQ”. É preciso configurar filas, mensagens, confirmações e consumidores. A FGV tende a explorar exageros: durable queue sozinha não basta; mensagem persistent sozinha não basta; ack não é publisher confirm; retry sem idempotência pode duplicar efeitos.

Mecanismo	O que faz	Limite / pegadinha
Durable queue	Fila sobrevive a reinício do broker.	Mensagens transitórias podem ser perdidas; durabilidade da fila não é tudo.
Persistent message	Mensagem marcada para persistência em disco.	Não elimina todas as janelas de falha; publisher confirms aumentam segurança.
Consumer ack	Consumidor confirma que processou a mensagem.	Não confirma publicação ao produtor.
Nack/reject	Consumidor rejeita mensagem, podendo reencaminhar ou dead-letter.	Requeue infinito pode gerar loop de falha.
Publisher confirm	Broker confirma ao produtor que aceitou a mensagem.	Não significa que consumidor processou o negócio.

Prefetch	Controla quantas mensagens não confirmadas podem ficar com consumidor.	Afeta fairness/carga, não é garantia de ordem global.
TTL	Tempo de vida de mensagem/fila.	Expiração não é sucesso de processamento.
DLX/DLQ	Destino de mensagens rejeitadas, expiradas ou com falhas.	DLQ é quarentena/investigação, não correção automática.

Fluxo com falha tratável:

1. API publica DocumentoJuntado na exchange eventos.processos.
2. fila.indexacao recebe a mensagem.
3. consumidor tenta indexar documento.
4. se serviço de busca está temporariamente fora, a mensagem pode ser reprocessada.
5. se exceder tentativas ou for inválida, vai para DLQ.
6. equipe corrige causa, reprocessa de forma controlada e idempotente.

10. Padrões de integração com mensageria

Padrão	Ideia	Quando aparece em prova
Work queue	Vários consumidores competem por mensagens de uma fila; cada mensagem vai para um consumidor.	Processar tarefas pesadas em paralelo.
Publish/Subscribe	Um evento é copiado para múltiplas filas/assinantes.	Notificação, auditoria e BI reagindo ao mesmo fato.
Event notification	Evento informa que algo mudou; consumidor busca detalhes se necessário.	Payload menor, mais desacoplado.
Event-carried state transfer	Evento carrega dados suficientes para o consumidor atualizar sua projeção.	Mais autonomia, mas risco de payload grande/versionamento.
Transactional outbox	Serviço grava estado e evento em tabela local na mesma transação; publicador envia ao broker depois.	Evitar inconsistência entre commit no banco e publicação de evento.
Idempotent consumer	Consumidor tolera mensagem duplicada sem repetir efeito indevido.	Necessário porque entregas duplicadas podem ocorrer.
DLQ + reprocessamento	Mensagens problemáticas ficam separadas para análise/retry controlado.	Evita perder erro silenciosamente ou travar fila principal.

A transactional outbox não está no edital por nome, mas é uma distinção de arquitetura importante: publicar evento e alterar banco na mesma operação distribuída é difícil. O padrão outbox evita que a aplicação confirme alteração no banco e perca o evento por falha de rede com o broker.

11. Ordenação, duplicidade e consistência eventual

Questões de prova podem sugerir que RabbitMQ garante ordenação global, exatamente uma entrega e consistência imediata entre todos os sistemas. Essas afirmações são perigosas. Em ambientes distribuídos, a entrega pode ser ao menos uma vez; consumidores podem falhar após executar parte do efeito; mensagens podem ser reenqueueadas; múltiplos consumidores aumentam paralelismo, mas dificultam ordem absoluta.

Tema	O que memorizar	Alternativa enganosa
At-most-once	Mensagem pode ser perdida, mas não duplicada.	Dizer que é o padrão desejável para evento crítico.
At-least-once	Mensagem tende a ser entregue uma ou mais vezes; exige idempotência.	Dizer que não há duplicidade.
Exactly-once	Em sistemas distribuídos é difícil e depende de camadas específicas; não é promessa genérica do RabbitMQ.	Dizer que RabbitMQ garante exatamente uma vez em qualquer cenário.
Ordem	Pode haver ordem em condições específicas, mas escala, múltiplas	Dizer que há ordenação global por

	filas/consumidores e retries complicam.	padrão.
Consistência eventual	Consumidores podem ficar temporariamente atrasados em relação ao produtor.	Dizer que todos os sistemas atualizam imediatamente.

12. RabbitMQ em uma arquitetura Angular + Java/Quarkus/Spring

Cenário: um sistema interno do TJSC possui frontend Angular, uma API Java de processos, uma API de documentos, serviço de notificação e serviço de auditoria. O usuário junta documento a um processo. A operação principal deve confirmar a juntada com segurança; notificação e indexação podem ocorrer depois.

Angular

|
| POST /processos/{id}/documentos

v

API processo-documento (Java/Quarkus ou Spring)

|-- grava documento e vínculo transacionalmente
|-- publica evento DocumentoJuntado

v

RabbitMQ Exchange eventos.processos

|--> fila.notificacao.documento --> consumidor de notificações
|--> fila.auditoria.processo --> consumidor de auditoria
|--> fila.indexacao.documento --> consumidor de busca/indexação

Leitura correta

A API principal não deve transferir para a fila uma validação essencial que precisa ocorrer antes da confirmação ao usuário. Já atividades pós-fato, independentes e tolerantes a atraso, como notificação e indexação, são bons candidatos a evento assíncrono.

13. Mensageria, PDPJ e integração no Judiciário

No contexto da PDPJ e de sistemas judiciais, eventos negociais são úteis porque muitos fatos podem interessar a múltiplos serviços: movimentação processual, juntada de documento, distribuição, comunicação de ato, confirmação de pagamento, alteração cadastral, geração de mandado e atualização de índice de busca. O broker evita acoplamento direto entre todos os produtores e consumidores.

Fato negocial	Possíveis consumidores	Por que evento ajuda
DocumentoJuntado	Notificação, auditoria, indexação, ciência das partes.	Múltiplos interessados reagem ao mesmo fato.
ProcessoDistribuido	Gabinete, pauta, estatística, BI.	Evita chamadas diretas e acopladas para cada sistema.
CustasPagas	Baixa financeira, movimentação processual, comunicação externa.	Integra sistemas com tempos diferentes.
AudienciaDesignada	Agenda, notificações, sala virtual, painel de acompanhamento.	Permite reações assíncronas e especializadas.

14. Trade-offs que a FGV pode explorar

Decisão	Benefício	Custo / risco
Usar RabbitMQ para eventos	Desacoplamento e processamento assíncrono.	Contrato, monitoramento, DLQ e idempotência.
Usar fila para tarefa pesada	Melhor tempo de resposta ao usuário e controle de carga.	Complexidade para acompanhar status.
Usar fanout para múltiplos consumidores	Um evento alimenta várias reações.	Pode gerar consumo duplicado se o desenho estiver errado.

Usar topic exchange	Roteamento flexível por padrões.	Padrões mal definidos causam mensagens perdidas ou indevidas.
Usar retry automático	Recupera falhas transitórias.	Loop de retry pode sobrecarregar sistema ou duplicar efeitos.
Usar DLQ	Isola mensagens problemáticas.	Não resolve a causa; precisa processo de análise/reprocessamento.
Aumentar consumidores	Mais throughput.	Pode afetar ordenação e concorrência.
Payload grande no evento	Consumidor fica mais autônomo.	Versionamento e custo de tráfego aumentam.

15. Pegadinhas FGV de alto rendimento

1. Mensageria não substitui API REST em todos os cenários; ela é forte para integração assíncrona, desacoplamento e eventos.
2. Evento comercial é fato ocorrido; comando é pedido de ação. Nome de evento costuma estar no passado.
3. Broker não é API Gateway: gateway recebe e roteia requisições HTTP; broker intermedeia mensagens.
4. Exchange não é fila: exchange roteia; fila armazena até consumo.
5. Direct não é broadcast; fanout é broadcast; topic usa padrões; headers usa cabeçalhos.
6. Ack do consumidor não é publisher confirm. Um confirma processamento; outro confirma aceite da publicação pelo broker.
7. Durable queue sem mensagem persistente não garante sobrevivência da mensagem.
8. DLQ não corrige erro de negócio; apenas isola mensagens problemáticas para análise e reprocessamento.
9. At-least-once pode gerar duplicidade; consumidor idempotente é essencial.
10. Mais consumidores aumentam vazão, mas podem complicar ordenação.
11. Mensagem em fila não significa que o negócio foi concluído.
12. RabbitMQ ajuda na resiliência, mas exige observabilidade, métricas, alarmes e governança de contratos.
13. Webhook e RabbitMQ não são a mesma coisa: webhook é chamada HTTP reversa; RabbitMQ é broker interno de mensagens.
14. Coreografia reduz orquestração central, mas dificulta rastreamento se não houver correlation ID e tracing.
15. Mensageria não resolve sozinha transação distribuída; padrões como outbox e idempotência são necessários.

15.1 Como escolher a topologia RabbitMQ em questões de cenário

A FGV pode narrar um cenário e pedir a solução mais adequada. O candidato que apenas decorou “fila” tende a errar, porque o ponto não é escolher RabbitMQ genericamente, mas definir se o fluxo é competição, distribuição para múltiplos consumidores, roteamento exato ou roteamento por padrão.

Cenário de prova	Topologia mais provável	Por quê
Vários workers equivalentes devem processar tarefas pesadas, cada tarefa uma única vez.	Uma fila com múltiplos consumidores concorrentes.	É padrão work queue: paralelismo por competição.
Um mesmo evento deve alimentar auditoria, notificação e BI.	Exchange fanout ou topic com filas separadas por consumidor.	Cada consumidor precisa receber sua cópia. Uma fila única competida faria apenas um deles receber cada mensagem.
Eventos têm chaves como processo.documento.juntado e processo.audiencia.designada.	Topic exchange.	Permite filtros por padrões: processo.#, processo.*.juntado.
Mensagem deve ir a uma fila específica por chave exata.	Direct exchange.	Roteamento por igualdade entre routing key e binding key.
Roteamento depende de metadados não expressos bem em string hierárquica.	Headers exchange.	Usa cabeçalhos e critérios de correspondência por metadados.

Armadilha clássica

Se três sistemas precisam receber o mesmo evento, não use uma única fila compartilhada por três consumidores diferentes, porque em uma fila competitiva cada mensagem é entregue a um consumidor. Para múltiplos interessados independentes, use filas separadas ligadas à exchange.

15.2 RabbitMQ não é Kafka: cuidado com a comparação implícita

Mesmo que o edital cite RabbitMQ, a FGV pode usar linguagem genérica de eventos e tentar confundir broker de filas com plataforma de streaming. Para esta aula, memorize: RabbitMQ é muito forte em roteamento, filas, work queues, pub/sub por exchanges e entrega para consumidores. Kafka, quando aparece em outros editais, costuma ser associado a log distribuído, retenção de eventos e replay por offset. O edital aqui nomeia RabbitMQ; portanto, a cobrança deve girar em torno de **exchange, fila, binding, routing key, ack, DLQ e broker de mensagens**.

Aspecto	RabbitMQ no edital	Comparação que pode enganar
Modelo mental	Mensagens roteadas para filas e consumidas.	Tratar como log de eventos para replay amplo por offset.
Ponto de prova	Exchange, queue, binding, routing key, ack, DLQ.	Cobrar partições e offsets como se fossem centrais no RabbitMQ.
Uso forte	Work queue, pub/sub, roteamento flexível, integração assíncrona.	Dizer que só serve para Big Data/stream analytics.
Risco	Fila sem consumidor acumula; consumidor não idempotente duplica efeito.	Achar que retenção/replay resolve tudo automaticamente.

15.3 Garantias: o que cada mecanismo realmente confirma

Uma das melhores formas de acertar questões difíceis é perguntar: “quem está confirmando o quê para quem?”. Em mensageria, há confirmações em pontos diferentes do fluxo.

Mecanismo	Quem confirma	Confirma o quê	Não confirma
Publisher confirm	Broker para produtor.	A mensagem foi aceita pelo broker, conforme regras de publicação.	Que o consumidor processou o negócio.
Mandatory return	Broker para produtor.	A mensagem não pôde ser roteada para fila quando mandatory=true.	Sucesso de consumo.
Consumer ack	Consumidor para broker.	A mensagem foi processada ou pode ser removida da fila.	Que a publicação foi transacionalmente atômica com o banco.
Nack/reject	Consumidor para broker.	Mensagem não foi processada com sucesso; pode requeue ou dead-letter.	Que o erro foi corrigido.
DLQ	Topologia do broker.	Mensagem foi isolada por rejeição, expiração ou política.	Que a mensagem deve ser descartada definitivamente.

15.4 Retry, DLQ e idempotência: desenho de falhas

Em prova, retry parece sempre bom. Na prática, retry sem limite pode derrubar o sistema. Mensagem inválida não vira válida só porque foi reenqueueada. Por isso, um desenho maduro separa falhas transitórias de falhas permanentes e define uma política de DLQ.

Situação	Resposta arquitetural adequada	Erro de prova
Serviço externo temporariamente indisponível.	Retry com backoff e limite; depois DLQ se persistir.	Reenqueue infinito imediato.
Payload inválido por contrato quebrado.	Nack sem reenqueue ou envio para DLQ com diagnóstico.	Tentar indefinidamente.
Consumidor caiu antes do ack.	Reentrega possível; consumidor precisa ser idempotente.	Assumir que nunca haverá duplicidade.
Mensagem na DLQ.	Analisar causa, corrigir	Excluir automaticamente sem rastreamento.

	dado/código/configuração e reprocessar com controle.	
Pico de mensagens.	Escalar consumidores e observar prefetch/latência.	Aumentar consumidores sem avaliar ordem e banco.

Exemplo de política conceitual de falha:

- Falha de rede no serviço de e-mail: retry limitado.
- Erro 400 por payload inválido: DLQ, pois retry não resolve.
- Timeout intermitente: retry com backoff.
- Violação de chave única por mensagem duplicada: consumidor idempotente deve reconhecer como já processado.

15.5 Idempotência na prática

Idempotência é a capacidade de executar uma operação uma ou várias vezes e chegar ao mesmo efeito final. Em mensageria, ela é essencial porque uma mensagem pode ser entregue novamente se o consumidor falhar antes do ack ou se houver retry após incerteza do resultado.

Técnica	Como ajuda	Exemplo
Identificador único do evento	Permite detectar mensagem já processada.	eventId em DocumentoJuntado.
Tabela de mensagens processadas	Registra eventId/consumer para evitar efeito duplicado.	ServicoNotificacao ignora eventId já tratado.
Operação de upsert	Atualiza/cria sem duplicar registro.	Criar projeção de processo em índice de busca.
Chave natural de negócio	Impede duplicidade por restrição do domínio.	Uma notificação por processo+evento+destinatário.
Ack após efeito concluído	Evita remover mensagem antes do processamento.	Consumidor só envia ack depois de gravar auditoria.

15.6 Contratos de mensagem: versionamento, schema e governança

Evento comercial também é contrato. Se o produtor altera campos, tipos ou semântica sem governança, consumidores podem falhar silenciosamente. **Em arquiteturas maduras, o contrato de mensagem precisa ser documentado, versionado e compatível com evolução.**

Boa prática	Por quê	Pegadinha
Nome de evento claro	Comunica semântica de domínio.	Usar nome técnico como msg001.
Versão de contrato	Permite evolução sem quebrar consumidores.	Alterar payload sem avisar.
Campos mínimos estáveis	Consumidores dependem de dados previsíveis.	Remover campo usado por consumidor.
Correlation ID / causation ID	Rastreia fluxo distribuído.	Achar que fila garante rastreabilidade sozinha.
Compatibilidade retroativa	Novos campos opcionais quebram menos.	Mudar tipo ou significado de campo obrigatório.

Exemplo conceitual de evento comercial:

```
{
  "eventId": "1b7f...",
  "eventType": "DocumentoJuntado",
  "version": "1.0",
  "occurredAt": "2026-06-08T10:15:00Z",
  "correlationId": "req-789",
  "payload": {
    "processoId": "12345",
    "documentoId": "987",
    "tipoDocumento": "peticao"
  }
}
```

```
}
}
```

15.7 Observabilidade de mensageria

Em arquitetura assíncrona, o usuário não vê diretamente cada falha. Por isso, a operação precisa observar filas, consumidores e DLQs. Uma fila crescendo pode significar pico normal, consumidor parado, erro em lote, gargalo no banco ou contrato quebrado.

Indicador	O que pode indicar	Ação típica
Tamanho da fila	Backlog, consumidores insuficientes ou falha de consumo.	Investigar taxa de publicação/consumo.
Taxa de entrada e saída	Produtor mais rápido que consumidor.	Escalar consumidores ou otimizar processamento.
Mensagens não confirmadas	Consumidor recebeu mas não deu ack.	Verificar travamento, prefetch e tempo de processamento.
Mensagens em DLQ	Falha permanente, contrato inválido ou retries esgotados.	Analisar causa e reprocessar com controle.
Tempo na fila	Latência assíncrona.	Ajustar capacidade e prioridade.
Erros por consumidor	Falha em integração, banco ou regra.	Corrigir código/configuração e proteger retries.

15.8 Antipadrões que a FGV pode transformar em alternativa

Usar RabbitMQ como banco definitivo de processos ou documentos.

Criar uma fila única para vários consumidores que deveriam receber cópias independentes do mesmo evento.

Fazer requeue infinito para mensagem inválida.

Colocar regra de negócio profunda no broker.

Publicar evento antes de confirmar a transação principal sem estratégia de consistência.

Consumidor sem idempotência em cenário de at-least-once.

Payload sem versão, sem eventId e sem correlationId em integração crítica.

Usar mensageria para consulta simples que exige resposta imediata.

Achar que DLQ é lixeira definitiva sem processo de análise.

Aumentar consumidores sem verificar lock, concorrência, ordem e capacidade do banco.

16. Questões autorais - padrão TJSC/FGV

Resolva as questões antes de consultar o gabarito. As questões são autorais e calibradas para discriminar conceitos próximos, sem reproduzir literalmente itens de prova.

Questão 1. Em um sistema distribuído, uma API publica uma mensagem para que outro serviço envie notificações posteriormente, sem que a chamada original precise aguardar o envio. A característica arquitetural mais diretamente explorada é:

- A) acoplamento transacional forte entre banco e consumidor.
- B) renderização server-side do frontend.
- C) comunicação assíncrona com desacoplamento temporal.
- D) descoberta dinâmica de serviços por DNS reverso.
- E) mapeamento objeto-relacional de entidades.

Questão 2. Em RabbitMQ, considerando o modelo AMQP, a peça que recebe mensagens publicadas e as encaminha para filas conforme tipo, bindings e routing key é denominada:

- A) Queue.
- B) Consumer.
- C) Virtual host.
- D) Exchange.
- E) Channel.

Questão 3. Analise as afirmativas. I. Evento comercial representa um fato relevante já ocorrido no domínio. II. Comando representa uma intenção de executar uma ação. III. Evento e comando são equivalentes, pois ambos sempre exigem resposta síncrona imediata. Está correto o que se afirma em:

- A) I, apenas.
- B) I e II, apenas.
- C) I e III, apenas.
- D) II e III, apenas.
- E) I, II e III.

Questão 4. Uma equipe propõe substituir todas as chamadas REST de consulta por RabbitMQ, inclusive buscas simples que exigem resposta imediata ao usuário. Sob a ótica arquitetural, a decisão é problemática porque:

- A) RabbitMQ não permite troca de mensagens entre aplicações Java.
- B) filas só podem ser usadas para persistência relacional.
- C) mensageria é proibida em microsserviços.
- D) RabbitMQ só funciona com frontend Angular.
- E) mensageria não é automaticamente superior a REST; consultas síncronas simples podem ser melhor atendidas por request/response.

Questão 5. No RabbitMQ, uma exchange do tipo fanout:

- A) encaminha cópia da mensagem para todas as filas vinculadas, ignorando a routing key.
- B) roteia apenas quando a routing key casa exatamente com a binding key.
- C) usa exclusivamente cabeçalhos para decidir destino.
- D) exige que exista apenas um consumidor por fila.
- E) armazena mensagens até que o produtor confirme o recebimento.

Questão 6. Um consumidor processa uma mensagem, grava o efeito de negócio e falha antes de enviar ack ao broker. Em um desenho com entrega ao menos uma vez, a consequência que exige maior cuidado é:

- A) a impossibilidade de usar filas duráveis.
- B) a conversão automática da mensagem em chamada REST.
- C) o risco de reentrega e duplicidade de efeito, exigindo idempotência.
- D) a perda obrigatória da fila inteira.

E) a transformação do exchange em API Gateway.

Questão 7. Relacione os conceitos. 1. API Gateway 2. Service Discovery 3. Message Broker 4. Banco de dados transacional () Entrada e roteamento HTTP com políticas transversais. () Registro/localização dinâmica de serviços. () Intermediação de mensagens e eventos. () Persistência do estado negocial. A sequência correta é:

- A) 3 - 2 - 1 - 4.
- B) 1 - 3 - 2 - 4.
- C) 2 - 1 - 3 - 4.
- D) 1 - 2 - 3 - 4.
- E) 4 - 2 - 3 - 1.

Questão 8. Sobre Dead Letter Queue/Dead Letter Exchange em RabbitMQ, assinale a alternativa correta.

- A) É mecanismo que garante exatamente uma entrega em qualquer cenário.
- B) É recurso para isolar mensagens rejeitadas, expiradas ou que não puderam ser processadas conforme regras, permitindo análise e reprocessamento controlado.
- C) É sinônimo de exchange fanout.
- D) É o componente responsável por autenticar usuários finais.
- E) É uma técnica para converter mensagens em tabelas JPA.

Questão 9. Considere as routing keys: processo.documento.juntado, processo.audiencia.designada e usuario.perfil.alterado. Para uma fila interessada em todos os eventos iniciados por processo, o padrão de topic exchange mais adequado é:

- A) processo.#
- B) processo.*.juntado
- C) usuario.#
- D) processo
- E) *.documento

Questão 10. Um produtor precisa saber se o broker aceitou a mensagem publicada. O mecanismo associado a essa confirmação, no RabbitMQ, é:

- A) consumer prefetch.
- B) DTO projection.
- C) lazy loading.
- D) service registry.
- E) publisher confirm.

Questão 11. Assinale a alternativa INCORRETA sobre mensageria em arquitetura de sistemas.

- A) Mensageria pode reduzir acoplamento temporal entre produtor e consumidor.
- B) Mensageria pode apoiar processamento assíncrono e suavização de picos de carga.
- C) Mensageria elimina a necessidade de contratos, versionamento, observabilidade e tratamento de erro.
- D) Mensageria pode permitir múltiplos consumidores reagindo ao mesmo evento.
- E) Mensageria pode exigir idempotência nos consumidores.

Questão 12. Uma aplicação publica o evento DocumentoJuntado. Três serviços independentes precisam reagir: auditoria, notificação e indexação. A configuração mais aderente ao padrão publish/subscribe em RabbitMQ é:

- A) uma única fila compartilhada pelos três serviços, de modo que apenas um deles processe cada evento.
- B) exchange fanout ou topic com filas separadas por consumidor interessado.
- C) banco único compartilhado entre os quatro serviços.
- D) requisição HTTP síncrona encadeada entre todos os consumidores.
- E) serviço de descoberta substituindo exchange e fila.

Questão 13. No contexto de RabbitMQ, prefetch está mais diretamente relacionado a:

- A) definir a URL pública do API Gateway.
- B) gerar automaticamente entidades JPA.

- C) alterar o formato JSON do payload.
- D) limitar a quantidade de mensagens não confirmadas entregues a um consumidor.
- E) substituir o uso de DLQ.

Questão 14. Um arquiteto afirma: “Se a fila é durável, qualquer mensagem publicada nela sobrevive a reinício do broker, independentemente das propriedades da mensagem”. A afirmação é:

- A) incorreta, pois a durabilidade da fila deve ser combinada com persistência da mensagem para maior garantia de sobrevivência.
- B) correta, pois durable queue sempre torna todas as mensagens persistentes.
- C) correta apenas quando a mensagem usa JSON.
- D) incorreta porque RabbitMQ não possui filas duráveis.
- E) correta apenas para filas de frontend.

Questão 15. Em uma arquitetura orientada a eventos, o padrão transactional outbox é usado principalmente para:

- A) substituir o broker por DNS.
- B) eliminar a necessidade de banco de dados.
- C) reduzir o risco de inconsistência entre alteração transacional no banco e publicação posterior do evento.
- D) obrigar todos os serviços a compartilhar a mesma tabela de mensagens.
- E) converter consumers em controllers REST.

Questão 16. A respeito de exchange direct, topic, fanout e headers, assinale a alternativa correta.

- A) Direct ignora routing key e envia para todas as filas.
- B) Fanout exige casamento exato entre routing key e binding key.
- C) Topic roteia apenas por cabeçalhos e nunca usa routing key.
- D) Headers é o único tipo que permite filas duráveis.
- E) Topic usa padrões de routing key, enquanto headers usa metadados/cabeçalhos.

Questão 17. Uma mensagem representa o fato “CustasPagas” e é consumida por serviço de movimentação processual e por serviço de BI. Em caso de falha temporária do BI, o desenho mais adequado deve:

- A) bloquear necessariamente a confirmação do pagamento até o BI voltar.
- B) prever retry, DLQ e idempotência, sem impedir que consumidores independentes processem de acordo com seus requisitos.
- C) transformar o evento em consulta GET.
- D) eliminar o broker e obrigar acesso direto ao banco do pagamento.
- E) garantir exatamente uma entrega sem necessidade de controle adicional.

Questão 18. Sobre webhook e RabbitMQ, assinale a leitura correta.

- A) Webhook e RabbitMQ são sinônimos de Service Discovery.
- B) RabbitMQ é uma API reversa em que o servidor chama o cliente por HTTP.
- C) Webhook substitui filas internas em qualquer cenário.
- D) Webhook é mecanismo de chamada HTTP para endpoint previamente cadastrado; RabbitMQ é broker de mensagens, frequentemente usado em comunicação interna assíncrona.
- E) Ambos são obrigatoriamente síncronos e bloqueantes.

Questão 19. Em uma questão da FGV, aparece o seguinte fluxo: API de processos publica ProcessoDistribuido; exchange eventos.processos roteia para fila.auditoria e fila.notificacao; cada fila possui consumidor próprio. A interpretação correta é:

- A) trata-se de polling, pois consumidores consultam periodicamente a API.
- B) trata-se de acesso direto compartilhado ao banco.
- C) trata-se de publicação de evento com múltiplos consumidores desacoplados por filas.
- D) trata-se de MVC, pois há Model, View e Controller.
- E) trata-se de ORM, pois há mapeamento objeto-relacional.

Questão 20. Assinale a alternativa INADEQUADA para consumidores de mensagens em integração crítica.

- A) Assumir que a mensagem nunca será entregue mais de uma vez.
- B) Projetar processamento idempotente.
- C) Registrar correlation ID para rastreabilidade.
- D) Tratar falhas transitórias com retry limitado.
- E) Encaminhar mensagens problemáticas para DLQ quando adequado.

17. Gabarito resumido

Questões
1C; 2D; 3B; 4E; 5A
6C; 7D; 8B; 9A; 10E
11C; 12B; 13D; 14A; 15C
16E; 17B; 18D; 19C; 20A

Gabarito em sequência: 1C - 2D - 3B - 4E - 5A - 6C - 7D - 8B - 9A - 10E - 11C - 12B - 13D - 14A - 15C - 16E - 17B - 18D - 19C - 20A

18. Comentários completos

Questão 1 - Gabarito: C. C é correta porque o cenário descreve desacoplamento temporal: o produtor não espera o processamento final. A fala de acoplamento forte, oposto da mensageria. B desloca para apresentação. D trata discovery, não mensageria. E trata persistência ORM.

Questão 2 - Gabarito: D. D é correta: a exchange recebe publicações e roteia para filas por tipo, binding e routing key. A fila armazena; consumer consome; virtual host isola; channel é canal lógico.

Questão 3 - Gabarito: B. I e II estão corretas. Evento é fato ocorrido; comando é pedido de ação. III erra ao igualar evento e comando e ainda exigir resposta síncrona, o que contraria o uso típico de eventos.

Questão 4 - Gabarito: E. E é correta porque mensageria não deve substituir automaticamente consultas síncronas simples. A, B, C e D são falsas ou caricaturais: RabbitMQ pode ser usado com Java, filas não são persistência relacional, mensageria é comum em microserviços e não depende de Angular.

Questão 5 - Gabarito: A. Fanout copia a mensagem para todas as filas vinculadas e ignora routing key. B descreve direct. C descreve headers. D não é requisito do fanout. E confunde exchange com fila/armazenamento.

Questão 6 - Gabarito: C. C é correta: se o consumidor executou o efeito e falhou antes do ack, a mensagem pode ser reentregue. Isso exige consumidor idempotente. As demais alternativas confundem fila, REST, perda total ou gateway.

Questão 7 - Gabarito: D. A sequência correta é API Gateway, Service Discovery, Message Broker e banco. As demais alternativas embaralham responsabilidades que a FGV costuma aproximar.

Questão 8 - Gabarito: B. DLQ/DLX isola mensagens rejeitadas, expiradas ou não processadas conforme política. A promete exatamente uma entrega sem base. C confunde com fanout. D é autenticação. E é ORM.

Questão 9 - Gabarito: A. processo.# captura zero ou mais segmentos após processo, incluindo processo.documento.juntado e processo.audiencia.designada. B é restrito demais. C filtra usuário. D não casa com segmentos. E captura apenas um segmento antes de documento.

Questão 10 - Gabarito: E. Publisher confirm confirma ao produtor que o broker aceitou a publicação. Prefetch controla consumo; DTO/lazy loading são outros temas; service registry é discovery.

Questão 11 - Gabarito: C. A incorreta é C. Mensageria não elimina contrato, versionamento, observabilidade nem tratamento de erro. A, B, D e E são características corretas ou cuidados necessários.

Questão 12 - Gabarito: B. Para três consumidores independentes reagirem ao mesmo evento, usa-se pub/sub com exchange fanout ou topic e filas separadas. A faria competição por uma mesma fila e apenas um serviço receberia cada mensagem. C, D e E confundem integração.

Questão 13 - Gabarito: D. Prefetch limita mensagens não confirmadas entregues a um consumidor. A é gateway; B é JPA; C é payload; E é DLQ.

Questão 14 - Gabarito: A. A afirmação é incorreta: fila durável ajuda a sobreviver ao reinício, mas mensagens também precisam ser persistentes para maior garantia. B generaliza indevidamente. C, D e E são falsas.

Questão 15 - Gabarito: C. Transactional outbox reduz o risco entre commit no banco e publicação de evento. Ele não substitui broker por DNS, não elimina banco, não obriga banco único e não transforma consumers em controllers.

Questão 16 - Gabarito: E. Topic roteia por padrões de routing key; headers por cabeçalhos/metadados. A e B invertem direct/fanout. C descreve headers como topic. D inventa exclusividade de filas duráveis.

Questão 17 - Gabarito: B. B é correta: consumidores independentes devem lidar com falhas por retry, DLQ e idempotência. A acopla indevidamente BI ao pagamento. C troca evento por consulta. D viola encapsulamento. E promete exatamente uma entrega sem controles.

Questão 18 - Gabarito: D. Webhook é chamada HTTP a endpoint cadastrado, funcionando como API reversa; RabbitMQ é broker de mensagens. A confunde com discovery. B chama RabbitMQ de API reversa. C exagera o webhook. E diz que ambos são síncronos/bloqueantes.

Questão 19 - Gabarito: C. O fluxo descreve evento publicado em exchange e distribuído a múltiplas filas/consumidores. A seria polling. B seria banco compartilhado. D é MVC. E é ORM.

Questão 20 - Gabarito: A. A é inadequada: em integração crítica não se deve assumir que mensagem nunca será entregue mais de uma vez. B, C, D e E são boas práticas: idempotência, correlação, retry limitado e DLQ.

19. Checklist final de cobertura da aula

Diferenciar mensageria de comunicação síncrona REST/HTTP.

Diferenciar evento negocial, comando, mensagem, consulta e notificação.

Entender o papel arquitetural do Message Broker.

Diferenciar broker, API Gateway, Service Discovery, banco e ESB.

Dominar o modelo RabbitMQ: producer, exchange, queue, binding, routing key e consumer.

Distinguir direct, fanout, topic, headers e default exchange.

Entender ack/nack, publisher confirms, durable queue, persistent message, TTL, DLQ/DLX e prefetch.

Reconhecer trade-offs: desacoplamento versus complexidade operacional; throughput versus ordem; retry versus duplicidade.

Saber que at-least-once exige idempotência e que exactly-once não é promessa genérica.

Aplicar mensageria a cenários de arquitetura e integração, inclusive PDPJ e sistemas corporativos.

20. Referências técnicas consultadas

Edital TJSC 2026 - Analista de Sistemas - FGV: <https://conhecimento.fgv.br/concursos/tjscservidor26>

Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.

RabbitMQ - AMQP 0-9-1 Model Explained: <https://www.rabbitmq.com/tutorials/amqp-concepts>

RabbitMQ - Exchanges: <https://www.rabbitmq.com/docs/exchanges>

RabbitMQ - Queues: <https://www.rabbitmq.com/docs/queues>

RabbitMQ - Consumers: <https://www.rabbitmq.com/docs/consumers>

RabbitMQ - Consumer Acknowledgements and Publisher Confirms: <https://www.rabbitmq.com/docs/confirmations>

RabbitMQ - Dead Letter Exchanges: <https://www.rabbitmq.com/docs/dlx>

RabbitMQ - Time-To-Live and Expiration: <https://www.rabbitmq.com/docs/ttl>

CloudEvents Specification: <https://cloudevents.io/>

AsyncAPI Specification: <https://www.asyncapi.com/docs/reference/specification/latest>

Spring guide - Messaging with RabbitMQ: <https://spring.io/guides/gs/messaging-rabbitmq>